16/32-bit Linear Executable Module Format 0:32 and Windows Virtual Device Driver (VxD) Model

—

Revision 1.0

Contents

Introduction

This document is intended to describe the interface that is used by language translators and generators as their intermediate output to the linker for the Microsoft OS/2 2.0 operating system. The linker will generate the executable module that is used by the loader to invoke the `*.EXE` and `*.DLL` programs at execution time.

This material was compiled from Microsoft OS/2 2.0 SDK sources, leaked Windows Debugger (short. `windbg`), Open Watcom documents and was written by Alexey Tolstopyatov the Tomsk State University student

Linear Executable File Format

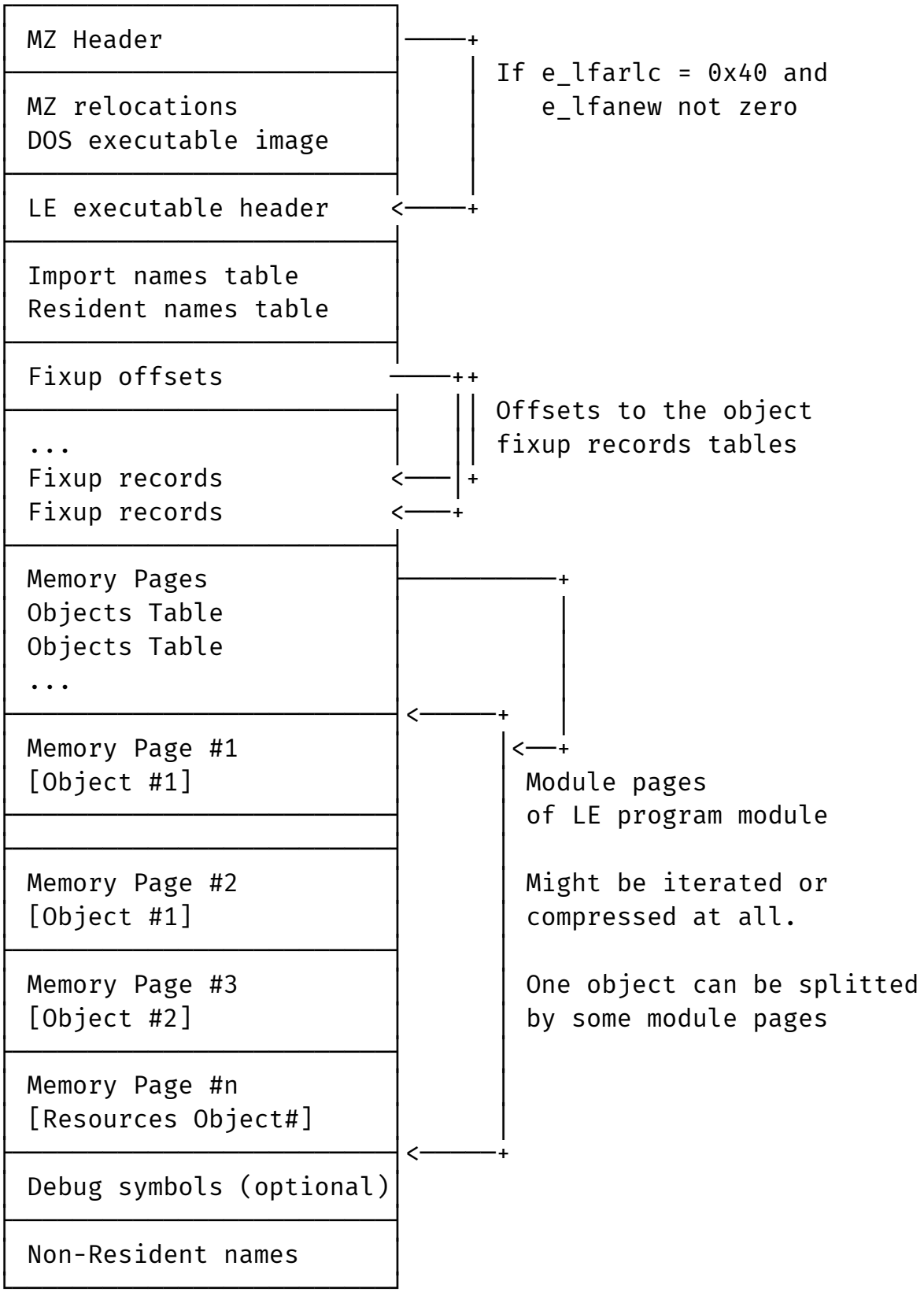
Linear Executable (short. `LE`) is file format for executable code designed for 16/32-bit protected mode operating systems. Originally used by the Microsoft OS/2 2.0 operating system, served as the file format for Virtual Device Drivers (`VxD`) in early versions of Windows, and adopted by various DOS extenders. Microsoft Windows 3.x and 9x used special archive format for `VxD` drivers – `W3` and `W4` executable containers.

> Do not use IBM's experience to determine the `LE` format correctly. The standard IBM OS/2 executable module format is a logical development of this model, which is not backward compatible!

Same with the New Executable format, first bytes of file belong to the Mark Zbikowski executable header. After those header bytes the DOS compatible program image starts. The field by the `0x3C` offset still hold an offset from the beginning of file to the next protected-mode executable image.

DOS 2.0 compatible part will skipped in this manual. First 2 bytes after `e_lfanew` long offset will be ASCII signature `[0x45, 0x4C]`. It has not been experimentally confirmed that the byte order affects the signature, however, I allow different byte order. That's why, is better to check `LE` and `EL` ASCII signatures.

At the next page presented file layout (see Listing 2)



Listing 2: Linear executable file layout

Despite the fact that `LE` format very similar with child – `LX` format, don't apply `LX` module tables to `LE` module format.

Since the IBM modification of linear executable module format, DOS 2.0 compatible program might be removed from file. So and `*.EXE/DLL/SYS` files will contain only `LX` program part.

Program Header

At this table presented full map of linear executable header

Offset	Size	Name	Description
0x00	2	e32_magic	Magic ASCII (0x454C)
0x02	1	e32_border	Byte ordering
0x03	1	e32_worder	Word ordering
0x04	4	e32_level	Format level (0 for initial version)
0x08	2	e32_cpu	CPU type
0x0A	2	e32_os	Operating system type
0x0C	4	e32_ver	Module version number
0x10	4	e32_mflags	Module flags
0x14	4	e32_mpages	Number of pages in module
0x18	4	e32_startobj	Object number for instruction pointer
0x1C	4	e32_eip	Extended instruction pointer
0x20	4	e32_stackobj	Object number for stack pointer
0x24	4	e32_esp	Extended stack pointer (offset within object)
0x28	4	e32_pagesize	Page size in bytes (default 4096)
0x2C	4	e32_lastpagesize	Size of last page in bytes
0x30	4	e32_fixupsize	Fixup section size in bytes
0x34	4	e32_fixupsum	Fixup section checksum
0x38	4	e32_ldrsize	Loader section size in bytes
0x3C	4	e32_ldrsum	Loader section checksum
0x40	4	e32_objtab	Offset of object table from start of LE header
0x44	4	e32_objcnt	Number of objects in module
0x48	4	e32_objmap	Offset of object page map

Offset	Size	Name	Description
0x4C	4	e32_itermap	Offset of object iterated data map
0x50	4	e32_rsrctab	Offset of resource table
0x54	4	e32_rsrccnt	Number of resource entries
0x58	4	e32_restab	Offset of resident name table
0x5C	4	e32_enttab	Offset of entry table
0x60	4	e32_dirtab	Offset of module directive table
0x64	4	e32_dircnt	Number of module directives
0x68	4	e32_fpagetab	Offset of fixup page table
0x6C	4	e32_frectab	Offset of fixup record table
0x70	4	e32_impmod	Offset of import module name table
0x74	4	e32_impmodcnt	Number of entries in import module name table
0x78	4	e32_impproc	Offset of import procedure name table
0x7C	4	e32_pagesum	Offset of per-page checksum table
0x80	4	e32_datapage	Offset of enumerated data pages
0x84	4	e32_preload	Number of preload pages
0x88	4	e32_nrestab	Offset of non-resident names table
0x8C	4	e32_cbnrestab	Size of non-resident name table in bytes
0x90	4	e32_nressum	Non-resident name table checksum
0x94	4	e32_autodata	Object number for automatic data object
0x98	4	e32_debuginfo	Offset of debugging information
0x9C	4	e32_debuglen	Length of debugging information in bytes
0xA0	4	e32_instpreload	Number of instance pages in preload section
0xA4	4	e32_instdemand	Number of instance pages in demand section
0xA8	4	e32_heapsize	Size of heap for 16-bit applications
0xAC	24	e32_res3	Reserved bytes to pad structure to 196 bytes

Total size of `LE` header – 196 bytes (or 0xC4). All offsets are from the start of it.

Differences between `LE` and `LX` headers starts here already. Field by the `0x2C` offset from the beginning of header doesn't store module pages alignment (short. `e32_pageshift`)

This difference is one of the key ones in the comparison. Further more. In the following sections, the layout of memory pages will be discussed, and this field will help.

Target CPU Type

Value	CPU Type
`0x01`	Intel 80286 or upwardly compatible
`0x02`	Intel 80386 or upwardly compatible
`0x03`	Intel 80486 or upwardly compatible
`0x04`	Intel Pentium or upwardly compatible

Table 1: CPU types. Values are stored in the `e32_cpu` field.

Target OS Type

Value	Description
`0x00`	Any “new format” OS
`0x01`	OS/2 2.0+
`0x02`	Windows
`0x03`	DOS 4.0
`0x04`	Windows/386

Table 2: OS types. Values are stored in the `e32_os` field.

Module Flags

This information bases on `exe386.h` header from MS OS/2 SDK. The module flags field `e32_mflags` divided on high and low `WORD` bitmasks. Empty cells at the high and low `WORD` columns mean zero value.

Bit(s)	Mask	Description
0	0x0001	Reserved
1	0x0002	Reserved
2	0x0004	Per-Process Library Initialization
3	0x0008	Reserved
4	0x0010	No Internal Fixups for Module
5	0x0020	No External Fixups for Module
6-7	0x00C0	Reserved
8	0x0100	Incompatible with PM Windowing
9	0x0200	Compatible with PM Windowing
10-11	0x0C00	Uses PM Windowing API
12	0x1000	Reserved
13	0x2000	Module not Loadable
14	0x4000	Reserved
15	0x8000	Library Module

Table 3: Low word of module flags.

Bit(s)	Mask	Description
0-14		Reserved
15	0x8000	.DLL module
16	0x10000	Protected memory library module
17	0x20000	Device driver
?	0x38000	Virtual `DLD` Driver
18	0x40000	Reserved
19-31		Reserved

Table 4: High word of `e32_mflags`

Then next listing demonstrates a way to define module flags from the given parsed header, (`e32_mflags` filled already).

```
mflags := 0x00000212;
pmcompat := mflags & 0x00000200;
isdll := mflags & 0x000008000;

if (pmcompat > 0) -> "Compatible with OS/2 PM";
if (isdll > 0) -> "Library module";
```

Listing 3: Resolving module flags bitmask routine

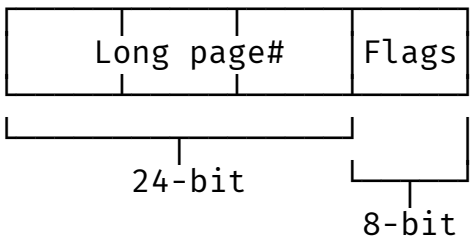
Remark

In fact, keep it simple when defining flags. There are no divisions into high and low `WORD`s in the original sources. This was done in order to divide the tables correctly. The Listing 3 shows it. In other regions of this document the large tables will be devided by this principle.

Object Page Map

Module pages concept defines the special “physical sections” of program text and data. At the moment of format revision `0:32` by IBM and Microsoft, module pages records are represent an array of 32-bit records.

On the next listing presented a single module page record.



Listing 4: Module page record layout

Numbering of module pages starts from one. Count of module pages contains in the executable header in the `e32_mpages` field. For each module page except last page the `e32_pagesize` is valid. Last module page have size which equal `e32_lastpagesize` value.

After the linkage the Module pages always aligned by the paragraph (`0x10`). To compute file offset to the page start, make sure that you know

- Long Page index;
- Page is vaid (`Flags` set to zero);
- Page size (`e32_pagesize`);
- Offset to enumerated datapages (`e32_datapage`);

```
page_offset := e32_datapage + (page# - 1) * e32_pagesize;
```

Per-page attributes or `Flags` field presented in the previous listing.

Byte	Description
0x00	Valid Physical Page
0x01	Iterated page
0x02	Invalid page
0x03	Zeroed page
0x04	Pages range

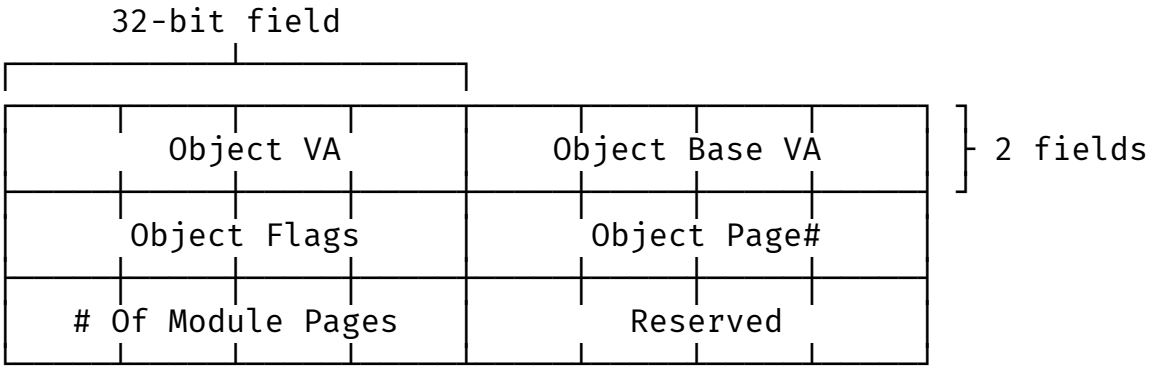
Table 5: Per-page attributes

Object Table

From the point of view of file, objects in the linear executable represents a reserved range of module pages where some text or data could be placed.

To find out which module pages reserved for which object, the file offset of the object table `e32_objtab` and `e32_objcnt` are read from the program header and entire table is reviewed.

Objects table is just an array of object records. Object record presented next



Listing 6: Object record layout

On this listing the `DWORD` fields ordering starts from upper left and reads till the least right field at the bottom.

Per-Object flags is a bitmask field which may contain bits which defined in the next table

Word	Description
0x0001	Readable Object
0x0002	Writable Object
0x0004	Executable Object
0x0008	Resource Object
0x0010	Object is discardable
0x0020	Object is shared
0x0040	Object has preload pages
0x0080	Object uses invalid pages

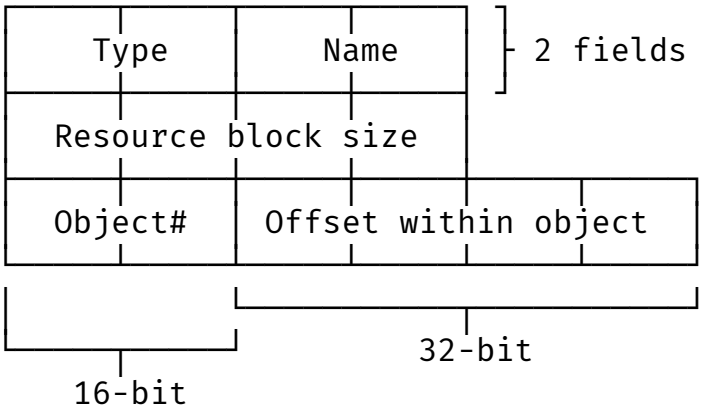
Table 6: Object flags (low byte)

Word	Description
0x0100	Object is permanent and swappable
0x0200	Object is permanent and resident
0x0300	Object is resident and contiguous
0x0400	Object is permanent and long locable
0x0600	Object is nonpermanent – should be
0x0700	Object type-mask
0x1000	16:16 alias required (80x86 specific)
0x2000	Big bit setting (80x86 specific)
0x4000	Object is conforming for code
0x8000	Object I/O Privileges

Table 7: Object flags (High byte)

Resource Table

The resource table is an array of resource table entries. Each resource table entry contains a `type` and `name`. These entries are used to locate resource objects contained in the Object table.



Listing 7: Resource Table record

The number of entries in the resource records is defined by the Resource Table Count located in the linear executable header. More than one resource may be contained within a single object.

On the next table presented available resource types for Microsoft OS/2 2.0

Byte	Description
0x01	Mouse pointer shape
0x02	Bitmap
0x03	Menu
0x04	Dialog
0x05	Strings Table
0x06	Font Directory
0x07	Font
0x08	Accelerator Table
0x09	Binary Data
0x0A	Message Tables (embedded `*.msg` file)
0x0B	Dialog include file name
0x0C	Key to `vkey` tables
0x0D	Key to UGL tables
0x0E	Glyph to character tables
0x0F	Screen display information
0x10	Function key area (short form)
0x11	Function key area (long form)
0x12	Help table
0x13	Help subtable
0x14	DBCS uniq/font driver directory
0x15	DBCS uniq/font driver

Table 8: Resource type ID definitions

Entry Point Table

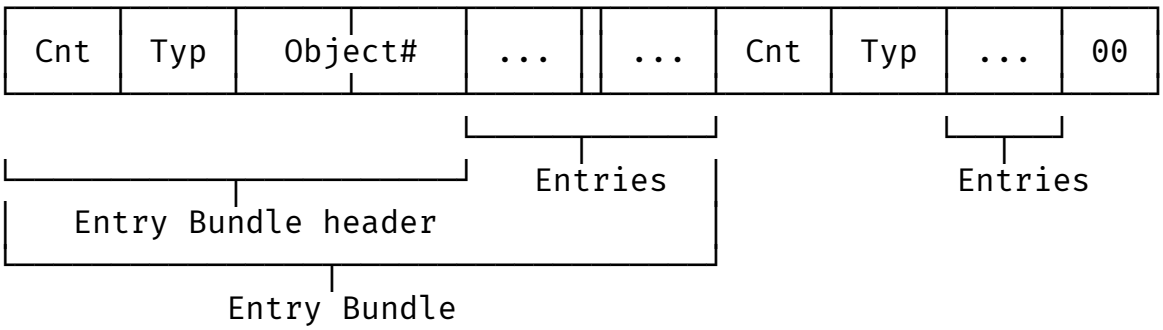
Entry points table (short. Entry Table) is very complex structure in the flat executable file format. Same with the segmented executables (short. `NE`) from OS/2 1.x and Windows 1.x, the entry table contains entry bundles.

Despite the fact that many different sources indicate the existence of only two types of entry bundles, the `exe_vxd.h` of leaked Windows debugger and `exe386.h` from Microsoft OS/2 2.0 SDK determines next types of entry point:

- Unused Entry;
- 16-bit Entry;
- 16-bit Callgate Entry;
- 32-bit Entry;
- Forwarder Entry;

```
type := typ & 0x7F;
has_add_typinfo := (typ & 0x80) != 0;
```

The bundle of entry points (short. Entry Bundle) contains data like an `object#` and type for all entry points which store in the current bundle. Next listing is a try to visualize a flat memory model of imaginary entry table.

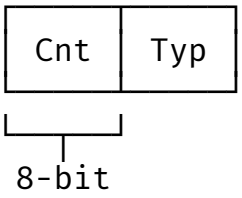


Listing 9: Entry Table layout

Entry Table starts with the `BYTE` which defines count of entry points in the given bundle. If entries count equals zero, this is a mark of the end of an Entry Table.

Entry Bundle for a Target object

Entry bundle header differs for other types of entries. If entry points bundle contains Forwarder entries – the target `object#` for entries in bundle ignores (doesn't exists at all). Linker and operating system sets `Object#` to zero.

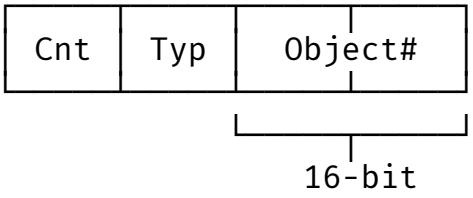


Listing 10: Forwarder or Unused Entry Bundle header

If `object#` exists and reads by the operating system loader, there're exist two ways.

- `object# = 0` then offset of each entry point in bundle are absolute file offset;
- `object#` not zero then each entry point (depending on the target object flags) have a virtual address!

Use Object Table and Module Pages Table to resolve the position of the entry point.



Listing 11: Entry Bundle header

All what is going next after header will be present an array of entry points with the type declared in the entry bundle header. Length of this array defines by the count field in the header (short. `Cnt`).

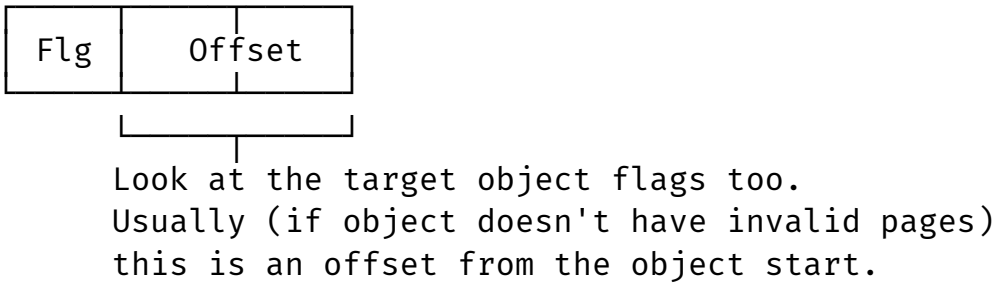
Entry Table represents a module exports which will be resolved during the another application run-time. Procedure ordinals enumeration starts from 1. Entry Point at the `0`-ordinal doesn't exist, but ordinal `0` exists and reserved by module.

Unused Entry

Unused Entry can't have a body instead of other types of enties. This is a way how to place export procedures `@1` and `@100`, for example. Linker skips the count of unused entries and starts new bundle.

16-bit Entry

After Entry Bundle, defined in previous region, follows an array of 16-bit entry points. For 16-bit program code entry point defines like on the next listing.



Listing 13: 16-bit Entry

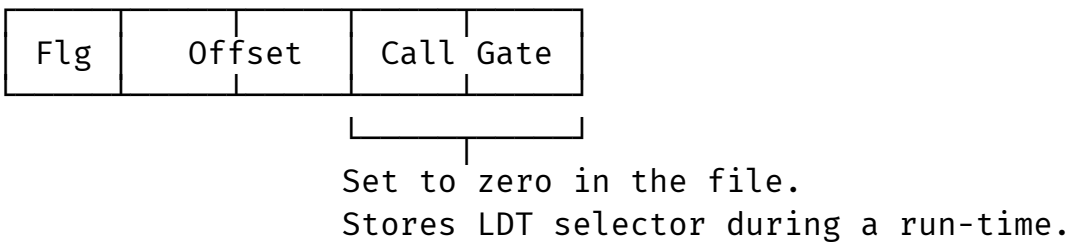
Flags `BYTE` field has a special bitmask.

Byte	Description
0x01	Exported Entry
0x02	Uses Shared `.DATA`
0xF8	Parameter work mask

Table 9: Entry Point record Flags

16-bit Call Gate Entry

The Intel 286 Call Gate Entry Point type is needed by the loader only if ring-2 segments are to be supported (Object flag bitmask includes `IO PRIVILEGIES` flag). The I286 Call Gate entries contain 2 extra bytes which are used by the loader to store an LDT callgate selector value



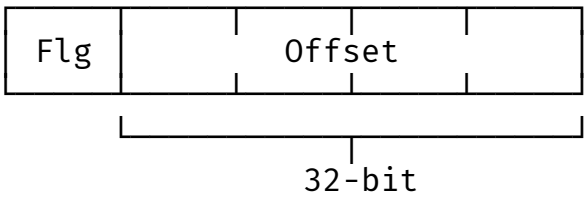
Listing 14: 16-bit Call Gate Entry

The DOSCALL1.DLL from the MS OS/2 stores I286 Call Gate entries. Examples of them will be the famous DosCopy procedure or DosRead/DosWrite procedures.

Flags of the Call Gate entry are the same with the 16-bit entry flags. So, see the Table 9.

32-bit Entry

Same with the 16-bit entries – 32-bit entries relate to objects with USE_32 flag in their bitmasks.



Listing 15: 32-bit Entry

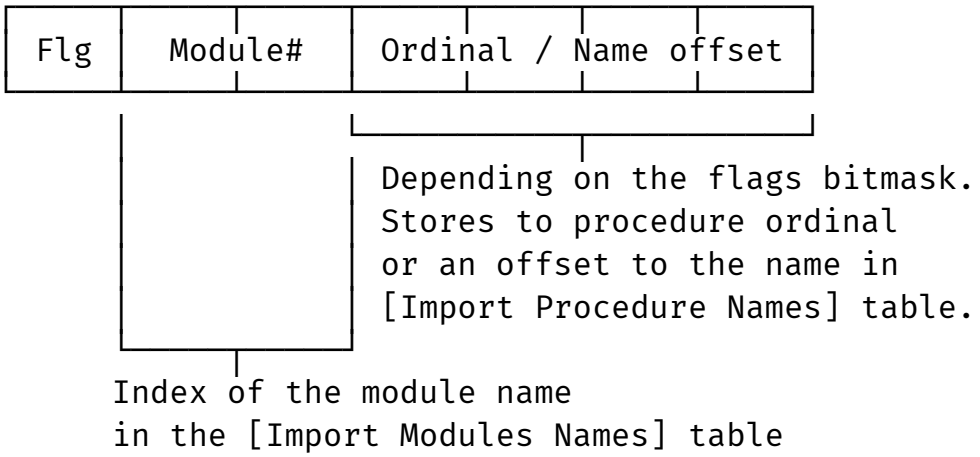
The Flg field contains same flags presented in the Table 9

Forwarder Entry

A Forwarder entry (short. forwarder) is an entry point whose value is an imported reference.

When a load time fixup occurs whose target is a forwarder, the loader obtains the address imported by the forwarder and uses that imported address to resolve the fixup.

A forwarder may refer to an entry point in another module which is itself a forwarder, so there can be a chain of forwarders. The loader will traverse the chain until it finds a non forwarded entry point which terminates the chain, and use this to resolve the original fixup. Circular chains are detected by the loader and result in a load time error.



Listing 17: Forwarder Entry record layout

For the forwarder entries the flags definitions are different.

Byte	Description
0x01	Import by ordinal
0xF7	Reserved!

Table 10: Forwarder Entry flags

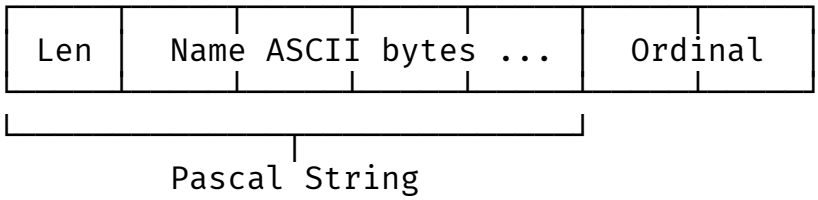
A maximum of 1024 `forwarders` is allowed in a chain; more than this results in a run-time time error.

Forwarders are useful for merging and recombining API calls into different sets of libraries, while maintaining compatibility with applications. *For example, if one wanted to combine* `MONCALLS`, `MOUCALLS`, *and* `VIOCALLS` *into a single libraries, one could provide entry points for the three libraries that are forwarders pointing to the common implementation.*

Resident Name Table

Resident names table contains export procedures which must be kept in the system memory during the module run-time. It is intended to contain the exported entry point names that are frequently dynamically linked to by name.

Resident name record consists of procedure ordinal word and pascal string of the procedure name.



Listing 18: Procedure Name record

The end of the table defines by the zero length of the next Pascal String.

Non-Resident Name Table

Non-resident names are not kept in memory and are read from the file when a dynamic link reference is made. Exported entry point names that are infrequently dynamically linked to by name or are commonly referenced by ordinal number should be placed in the non-resident name table.

Non-Resident names Table pointer in the Linear Executable header declares the offset from the start of file! This is a once structure in the LE layout which location defines not from start of program header.

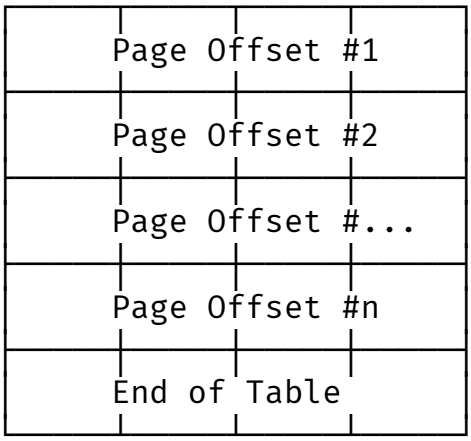
Same with the Resident Names table – it just an array of records (see Listing 18) with undefined count of entries.

The end of the table defines by the zero length of the next Pascal String.

Fixup Page Table

The fixup records table are array of fixup records and offset into fixup records are relative to the current page. Fixup page table serves to identify fixup records into code and data segments offset.

Fixup page table starts by the `e32_fpagetab` offset from a start of `LE` header and presents an array of 32-bit unsigned integers. Count of fixup pages is a `pages# + 1`.



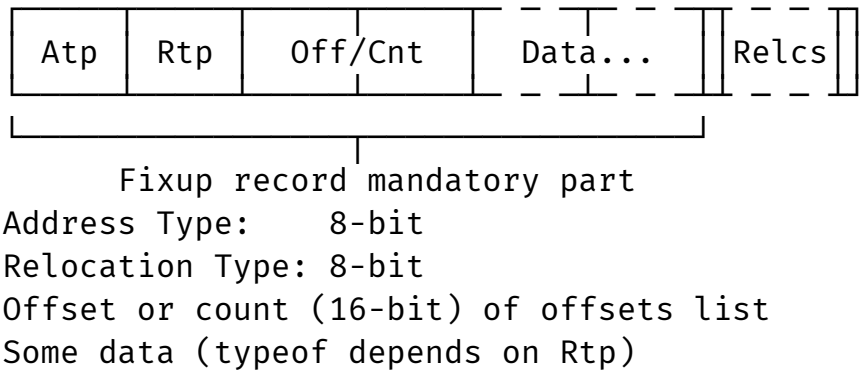
Listing 19: Fixup Page table

Each `DWORD` contains offset into Fixup Record Table of first fixup in the current page. Last `DWORD` contains size of fixup record table in bytes. I.e. substraction contains `(DWORD + 1)` with current `DWORD` is fixup table size for current page.

Fixup Record Table

Fixup Record table is the most complex structure in the `LE` executable. Starts by the `e32_frectab` offset from the start of a Program Header.

The first byte of each fixup record defines the address type `Atp` and source type `Rtp`




```
Address Type:    8-bit
Relocation Type: 8-bit
Offset or count (16-bit) of offsets list
Some data (typeof depends on Rtp)
```

Listing 20: Fixup Record template

The following next table presents an Address Type (short. `Atp`) possible definitions.

Atp value	Description	Size
0x00	8-bit fixup	1 byte
0x02	16-bit selector (segment)	2 bytes
0x03	16:16 far pointer (selector + offset)	4 bytes
0x05	16-bit offset	2 bytes
0x06	16:32 far pointer (selector + offset)	6 bytes
0x07	32-bit offset	4 bytes
0x08	32-bit self-relative offset	4 bytes

Table 11: `Atp` definitions. Values are stored in the low 4 bits of the first byte of a fixup record.

Also, the `Atp` field contains information continuation of current record:

Mask (Atp & …)	Meaning when set	Meaning when clear
0x10	Fixup to 16:16 alias (target object requires alias)	Normal fixup
0x20	The `Off/Cnt` field contains a count of following offsets	The `Off/Cnt` field is a single offset

Table 12: Additional `Atp` flags (bits 4 – 5 of the first byte).

After the source type follows the relocation type byte (short. `Rtp`). Full available definitions presented in the next table.

Rtp & 0x03	Description
0x00	Internal reference
0x01	Import by ordinal
0x02	Import by name
0x03	Fixup via Entry Table

Table 13: `Rtp` definitions. Stored in bits 0 – 1 of the `nr_flags` field.

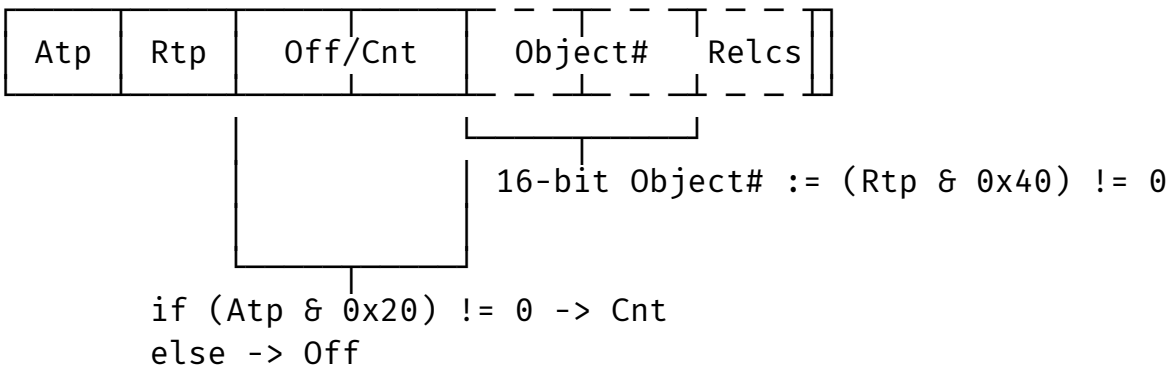
The relocation type field contains a special additive flag, which completely defines the layout of the fixup record.

`(Rtp & ...) != 0`	**True**	**False**
0x04	An additive value trails the fixup record before the offset list	The additive value is missing. (Doesn't exists in the record)
0x10	32-bit Target offset	16-bit target offset
0x20	32-bit Additive fixup flag	16-bit Additive fixup flag
0x40	16-bit `object#` or a `module#`	8-bit `object#` or a `module#`
0x80	8-bit procedure ordinal	16-bit procedure ordinal

Table 14: Relocation Type data sizes

Internal Fixup record

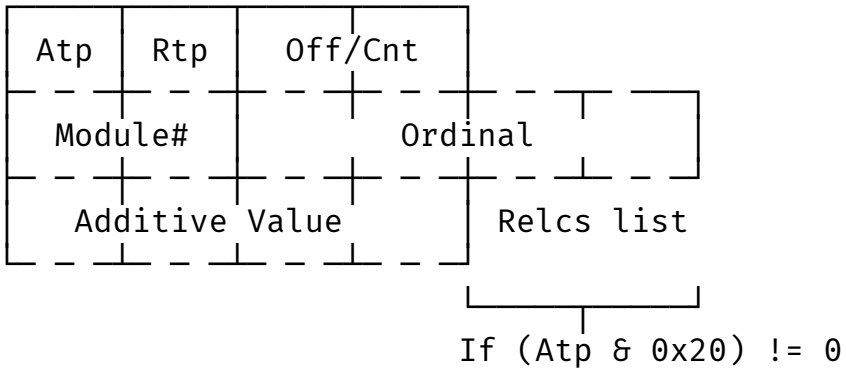
If the `Rtp` field conjuncted by type mask returns zero (see Table 13) the layout of Internal fixup record still depends on resolved `Atp` and `Rtp` masks.



Listing 21: Internal Fixup

Import by ordinal Fixup Record

The Import fixup record data differs with the record if it would be internal reference.



Listing 22: Ordinal Import Fixup

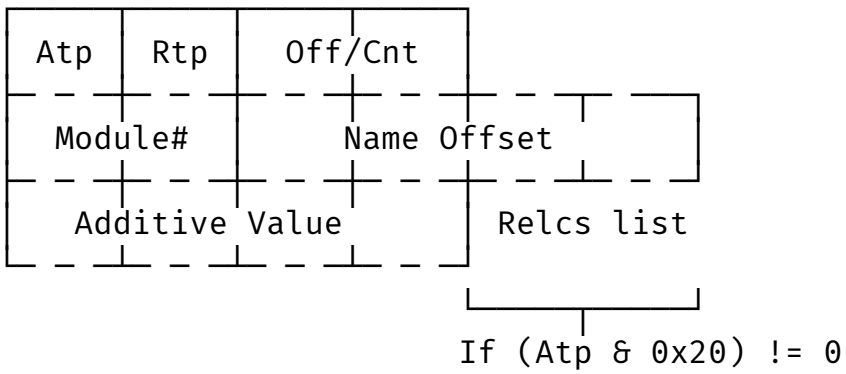
Memory layout objects on the Listing 22 which was drawn not by the “solid line” are optional or having variant size which is depending on the `Atp` and `Rtp` masks.

- `Module#` might be 8-bit or 16-bit;
- `Ordinal` might be 8-bit or 16-bit or 32-bit;

If the Additive Flag is set in the `Rtp` bitmask, the “Additive Value” (see Listing 22) is added to the address derived from the target entry point. This field is a `WORD` value when the “32-bit Additive Flag” bit in the target flags field is clear and a `DWORD` value when the bit is set.

Import by name Fixup Record

The fixup record of runtime import by name looks like the fixup of anonymous runtime import (see Listing 22).



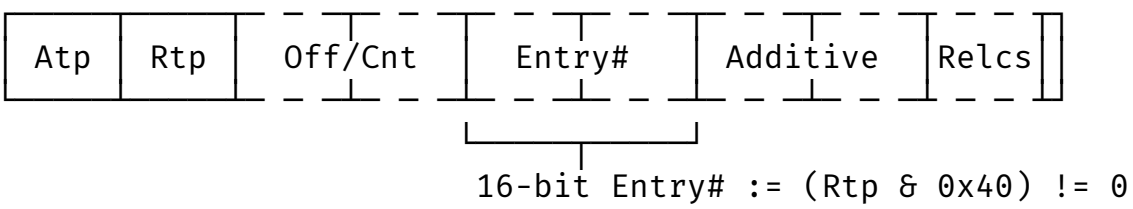
Listing 23: Import by Name Fixup

The `Name Offset` field is an offset into the Import Procedure Name Table. It is a 16-bit value when the 32-bit `Rtp` Flag bit in the target flags field is clear and a 32-bit value when the bit is set.

Other characteristics except the `Name Offset` define the same way like an Import by ordinal.

Entry Table Fixup Record

The Entry Table fixup (or a “Fixup via Entry Table”) record in the table makes the point for the loader that entry point by following ordinal must be relocated during the run-time.



Listing 24: Fixup via Entry Table record

The fixup via entry table record applies to the local Entry Table. If Entry Table of the module is empty but fixup records still exists, there’s a problems due the parsing of Entry Table or Fixup record Table.

The `Entry#` or an ordinal of given entry point might be 8 or 16-bit.

Import Module Table

The Import module table stores in the Linear Executable program/library by the `e32_impmodtab` offset from the beginning of the `LE` header. It contains symbols of run-time import programs or libraries, not functions/procedures.

The import module table is just an array of Pascal-Strings with fixed length which is defined in the `LE` header (see `e32_impmodcnt`)

The Import Module Table doesn't use the names of file system objects (i.e. programs or libraries from the point of view of the file system)!

Symbols what presented here are records of Resident Name Table **by the zero ordinal** of each module which entry points will be used by the current module during the run-time.

Also knowing of the `e32_impmodcnt` value helps to avoid tables overlap problems. If the DLL which contains just Forwarder Entries and no more code and data, the padding between the tables could be missed. By this reason, You may read Import module table and Resident Name table as a single table with a corrupted strings because of Ordinal records in the Resident Names table.

Import Procedure Table

Same with the Import Module table the Import Procedure table looks like an array of Pascal Strings. But the end of the table defines by the last Pascal-String.

Runtime Imports Symbols remark

The Import Procedure table contains case-sensitive Pascal-strings like resident names table (see Listing 18). But Microsoft `LNK386.EXE` follows the same algorithm with the `LINK.EXE` for 16-bit segmented programs ignores the case of module API. That's why after the resolving of symbols you will get the `INVALIDCASE` named list of DLLs and executables.

In the modern IBM `LX`-linked modules which provide generic/template API or require them, you will see the Itanium ABI-mangled symbols. And they're case sensitive. This fact is a proof of `LE` and `LX` symbols storage concept.

Windows Virtual Device Driver Model

"VxD" is the device driver model used in the 386 enhanced mode of Windows 3.x, Windows 9x, and to some extent also by the Novell DOS 7, OpenDOS 7.01, and DR-DOS 7.02 (and higher) multitasker (TASKMGR).

VxDs have access to the memory of the kernel and all running processes, as well as raw access to the hardware. Starting with Windows 98, Windows Driver Model was the recommended driver model to write drivers for, with the VxD driver model still being supported for backward compatibility, until Windows Me.

Also the virtual device drivers which built using this model contains a few parts of code:

- 16-bit Intel Real mode initialization code object;
- 32-bit Intel Protected mode initialization code object;
- 32-bit locked/unlocked code/data objects;
- Device Declaration Block (short. `DDB`)

And no iterated or invalid data pages at all.

The 16-bit real mode code object contains program text which has been written in the `VxD_REAL_MODE_INIT_SEG` macro, provided by the `VMM.inc`. This code which round of macro opening and closing parts will become an `ICOD` object with the `16:16 Alias` and `USE_16` attributes.

The 32-bit initialization code and data segments defined by the `VxD_ICODE_SEG` and `VxD_IDATA_SEG` macros are present until `VMM` has completed initialization, at which time they are discarded, freeing the memory used by these sometimes cumbersome pieces of code. These initialization procedures can determine whether it is safe to load the VxD or to bail out prior to further initialization. Thus, the VxD load can fail, the user can be notified, and there will be no memory wasted for the VxD when the VMM completes initialization.

The Locked code and data objects will become a `LCOD` and data objects with the `USE_32` attribute.

Virtual Device Driver Header

At the end of Linear Executable header in the padding field (short. `e32_res3`) embeds the device driver header.

Type	Value
`DWORD`	e32_winresoff
`DWORD`	e32_winreslen
`WORD`	e32_devid
`WORD`	e32_ddkver

Table 15: VxD Header

The `e32_winresoff` (optional) contains an offset from the beginning of file to the special information block and compiled Windows resource `*.res`. Don't use Resource Table definitions of `LE` module.

The `e32_winreslen` field tells the length of compiled Windows resource file and special VxD Resource structure.

The `e32_devid` presents a Device ID for VxD and `e32_ddkver` presents a version of Windows Device Driver Kit. Also you can read this `WORD` as an array of two `BYTE`s.

Virtual Device Resource Block

The resource block of VxD presents the next structure which defines two important values: the compiled resource file embedded into the `LE` module image and the ordinal of Device Declaration Block entry point.

Type	Value
BYTE	type
WORD	id
BYTE	name
WORD	ordinal
WORD	flags
DWORD	size

Table 16: VxD Resource block

According to personal assumptions, using the Ordinal field, you can find the number of the DDB entry point. I have no good reason to believe that this is correct, but I don't see any alternatives to determine the ordinal of the entry point.

Device Declaration Block

The device declaration block describes the virtual device to the `VMM`. It provides a VxD mnemonic, usually a somewhat descriptive title using `V` as the prefix and `D` as the suffix, such as `VJOYD`, suggesting a virtual joystick driver for example.

It also provides a major and minor version, the main control procedure, the device ID number, the initialization order, and control procedures for the V86 or Protected-Mode API.

Type	Value	Default	Description
DWORD	ddb_next	0	VMM reserved
WORD	ddb_version		DDK version
WORD	ddb_dev_id	UNDEFINED_DEVICE_ID	Required Device ID
BYTE	ddb_dev_major	0	Major device number
BYTE	ddb_dev_minor	0	Minor device number
WORD	ddb_flags	0	
BYTE[8]	ddb_name	“ “ (8 spaces)	VMM device mnemonic
DWORD	ddb_init_order	UNDEFINED_INIT_ORDER	The load order
DWORD	ddb_ctrl_offset		_Control_Proc offset
DWORD	ddb_v86api_offset	0	Offset to API procedure
DWORD	ddb_pmapi_offset	0	_Device_Init offset
DWORD	ddb_v86api_csip	0	CS:IP of API entry point
DWORD	ddb_pmapi_csip	0	CS:IP of API entry point
DWORD	ddb_rmdata_ref	0	Real mode data reference
DWORD	ddb_srvtab_ptr	0	Pointer to service table
DWORD	ddb_srvtab_size	0	Number of services
DWORD	ddb_win32tab_ptr	0	Win32 services table
DWORD	ddb_prev4	4.0	Pointer to prev 4.0 DDB
DWORD	ddb_res0	0	Reserved
DWORD	ddb_res1		Reserved
DWORD	ddb_res2		Reserved
DWORD	ddb_res3		Reserved

Table 17: VxD Device Declaration block

The field ddb_flags at the moment of writing is still unknown. Far pointers like ddb_v86api_csip and ddb_pmapi_csip are set to NULL and later fills by the operating system during the run-time.

Are VxD Drivers are 32-bit or 16-bit?

Answer for this question hides in the VMM.inc and whole Windows DDK insights. If you try to disassemble a few virtual drivers you will see that far pointer to the entry point always points to the 16-bit .CODE object.

This entry point uses by `VMM` to wake up device driver. Depending on Windows DDK macros might be different.

```
; Object# : 3.
; Pages#  : 1 (present in the file)
; Virtual Size : 0x00000084
; Flags (0x00001005): [Readable] [Executable] [16:16 Alias
Required] [USE_16]
; Segment type: Pure code
; segment para public .CODE
assume cs:03
assume es:03, ss:00, ds:03, fs:00, gs:00

03:0000                 public start
03:0000 start           proc near
03:0000                 cmp     bx, 1
03:0003                 jz      loc_C0007016
03:0007                 mov     ax, 0
03:000A                 call    sub_C0007024
03:000D                 jb      loc_C0007016
03:0011
03:0011 loc_C0007011: ; CODE XREF: start+19↓j
03:0011                 xor     bx, bx
03:0013                 xor     si, si
03:0015                 retn
03:0016
03:0016 loc_C0007016: ; CODE XREF: start+3↑j
03:0016 ; start+D↑j
03:0016                 mov     ax, 8001h
03:0019                 jmp     short loc_C0007011
03:0019 start           endp
```

Listing 25: 16-bit initialization code fragment

But the Device Declaration block stores in the 32-bit code objects. The `Control_Proc` stores in the locked 32-bit code object. The `Dev_Init_Proc` and `V86_Proc` might be store in the 32-bit or 16-bit code objects.

As you see, no 286 Call Gate entries here because of Device driver API presented in the hidden Declaration block and VMM operates loaded driver using this table. The DDB Entry Point always be 32-bit entry because of near addresses of driver API stores together.

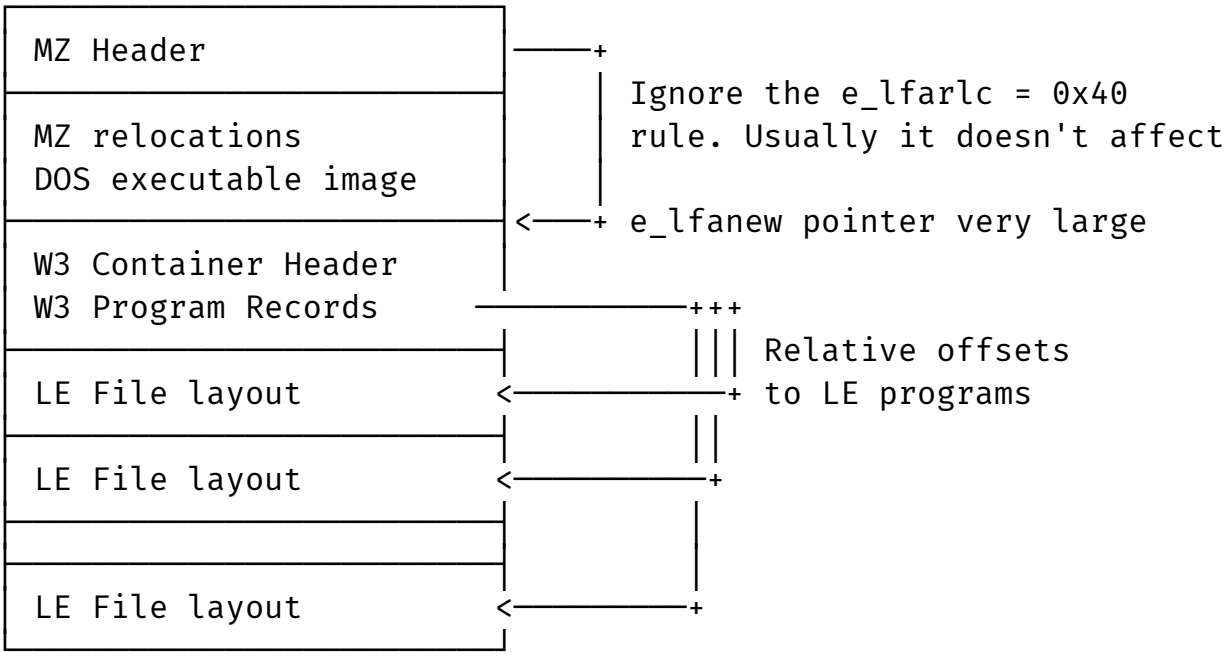
The W3 Container Format

This region wasn't declared in the title of this research work but W3 and W4 formats silently exists in the Microsoft Windows 3.x and 9x.

Much of information in this region was taken from the "Undocumented Windows Formats" book by Pete Davis and Mike Wallace.

W3 File Layout

On the Listing 26 presented file layout and as you can see, this file is very large. Usually in the Windows 3.x this container uses for `VMM.386`.



Listing 26: W3 file layout

Unfortunately, drivers within a `W3` file are somewhat mangled, although the mangling is fairly minor and easy to rectify. First of all, the `e32_datapage` value of `LE` header is changed to be relative to the beginning of the `MZ` header for the `W3` file. Normally this is relative to the beginning of the `MZ` header of the `LE` file. Of course, in a `W3` file, the stubs for virtual drivers have been stripped to save space.

The changing of the `e32_datapage` value is weird though, because none of the other offset fields are changed at all. The other mangling has to do with the Non-Resident name table.

The Non-Resident name table is usually the last section of an LE file. In the case of W3 files, however, all Non-Resident name tables have been removed, so if you extract the LE files, you'll need to build one for it. This is a fairly simple process, however.

W3 Header

W3 header presented on the Listing 26 looks pretty simple. On the Table 18 presented full header of the W3 container

Type	Name	Value
`BYTE[2]`	`w3_magic`	ASCII "W3" signature
`WORD`	`w3_winver`	Windows version
`WORD`	`w3_modcnt`	Number of programs in the container
`BYTE[10]`	`w3_res`	Padding
`W3REC[N]`	`w3_moddir`	W3 records array

Table 18: W3 Header layout

After the W3 header follows the array of `W3REC` records. The length of this array defines by the `w3_modcnt` field.

Type	Name	Value
`BYTE[8]`	`w3_modname`	Fixed 8-byte string (no zero terminated)
`DWORD`	`w3_modhdr`	Offset to the module LE header
`DWORD`	`w3_cbmodhdr`	Count bytes in the module header

Table 19: W3 Record

The W4 Container Format

The W4 file format is very similar to the W3 file format, except once. The W4 container uses a special compression algorithm. In fact, the compression used in the W4 file is exactly the same as the Double Space compression used in double-space drives.

The W4 compression algorithm is called a Lempel/Ziv/Welch (short. LZW) compression algorithm“, named after the three people who contributed to its design. The W4 file, when decompressed, actually contains a W3 file inside, so once you’ve decompressed the W4 file, you can traverse the W3 file structure described earlier (see Listing 26).

In the Appendix A presented the decompression algorithm.

W4 Header

Type	Name	Value
BYTE[2]	w4_magic	ASCII “W4” signature
WORD	w4_res1	Reserved
WORD	w4_cbchunk	Chunk size
WORD	w4_chunkcnt	Count of chunks
BYTE[2]	w4_dsmag	ASCII “DS” magic
BYTE[6]	w4_res3	Usually NULLs

Table 20: W4 Header layout

W4 Decompression

So, instead of explain ing what a Shannon-Fano table is, I will only go into how it specifically affects W4 files. At its most basic, the Shannon-Fano table provides codes that give the depth and count of repeated data in the compressed data. The Shannon-Fano table for the W4 compression algorithm is shown in Table 21, Table 22 and Table 23

MSB...LSB	Description
xxxx01	1xxxx uncompressed byte
xxxx10	0xxxx Uncompressed byte

Table 21: Shannon-Fano Table

MSB...LSB	Description
00000000	Exit Code
xxxxxx00	xxxxxx = 1 - 63
11111100	63
xxxxxxxx011	64 + xxxxxxxx = 64 - 319
xxxxxxxxxxxx111	320 + xxxxxxxxxxxx = 320 - 4414
111111111111111	(4415) = Check Buffer

Table 22: Depth definitions

MSB...LSB	Description
1	2
010	3
110	4
xx100	5 + xx = 5 - 8
xxx1000	9 + xxx = 9 - 16
xxxx10000	17 + xxxx = 17 - 32
xxxxx100000	33 + xxxxx = 33 - 64
xxxxxx1000000	65 + xxxxxx = 65 - 128
xxxxxxx10000000	129 + xxxxxxx = 129 - 256
xxxxxxxx100000000	257 + xxxxxxxx = 257 - 512
000000000	Done

Table 23: Count definitions

The idea of how the Shannon-Fano table works is quite simple. At this point, it's probably best just to examine the code in Appendix A; in particular, `w4_extract()` and `load_buffer()`. Notice that `w4_extract()` pulls only as many bits from dwMiniBuffer as it needs. After pulling a depth value, it calls `load_buffer()` to shift in some new bits. Then it looks for the count value, again pulling only as many bits as it needs, and then calling `load_buffer()` to fill up `buffer` again.

In the End

In this document, the format of Linear Executable files (i.e. files using a flat memory model) was almost completely described. Serious topics about the device of virtual drivers were also touched upon and evidence was provided about their different nature (mixed 16 and 32-bit code).

Sections about the W3 and W4 containers are the last words in the direction of the VxD drivers, since Microsoft basically put them away and hid them in this way.

I really hope this work will save an unrealistic amount of research time and guide those who are trying to understand or simply explore this ambiguous period in the history of IBM and Microsoft corporations, or are trying to disassemble `LE`-linked modules.

Sources

- Open Watcom 1.8 and 2.0 sources;
- Microsoft OS/2 2.0 SDK (exactly `exe836.h`);
- IBM OS/2 2.0 diskette;
- Windows Device Driver Kit;
- Microsoft Windows Debugger for NT 3.5
- “Undocumented Windows Formats” book by Pete Davis
- “Writing Windows Device Drivers” article

Appendix A

```
#include <stdio.h>
#include <stdlib.h>
#include <memory>
#include <string.h>

void load_buffer(DWORD* buff, BYTE** src, WORD* used, count) {
  buff >>= 1;
  while ((*used)--) {
    buff += (DWORD) **src << 24L;
    *src += 1;
    *count = 8;
  }
}
/// Decompress the W4 Container
/// \param src given source bytes
/// \param dst destination file bytes
/// \param size safe length of given source
void w4_extract(BYTE* src, BYTE* dst, WORD size) {
  DWORD buffer = 0;
  WORD count;         // Count and Depth of compressed "string"
  WORD used;          // Count of bits used by the last "code"
  WORD depth;
  WORD dsti;          // Destanation index
  WORD temps = size; // Temporary size
  WORD i;             // Global iterator

  BYTE* tempb = src;
  // Load up the mini buffer (buffer) with the first 4 bytes.
  // Expecting something like that
  // msb             buffer            lsb
  // +--------+--------+--------+--------+
  // | BYTE 3 | BYTE 2 | BYTE 1 | BYTE 0 |
  // +--------+--------+--------+--------+
  dsti = 0;
  for (i = 0; i <= 3; i++)
    buffer = (buffer >> 8) + ((DWORD)*src++ << 24);

  count = 8;
  depth = 1;
```

```
  while (depth) {
    // Too many conditions will be written later
  }
}
```